

OS-Oracle: A Comprehensive Framework for Cross-Platform GUI Critic Models

Zhenyu Wu^{1,2*}, Jingjing Xie^{3*}, Zehao Li², Bowen Yang², Qiushi Sun⁴, Zhaoyang Liu⁵,
 Zhoumianze Liu², Yu Qiao², Xiangyu Yue³, Zun Wang², Zichen Ding^{2†}

¹Shanghai Jiaotong University ²Shanghai AI Laboratory ³CUHK MMLab

⁴The University of Hong Kong ⁵The Hong Kong University of Science and Technology

Abstract

With VLM-powered computer-using agents (CUAs) becoming increasingly capable at graphical user interface (GUI) navigation and manipulation, reliable step-level decision-making has emerged as a key bottleneck for real-world deployment. In long-horizon workflows, errors accumulate quickly and irreversible actions can cause unintended consequences, motivating critic models that assess each action before execution. While critic models offer a promising solution, their effectiveness is hindered by the lack of diverse, high-quality GUI feedback data and public critic benchmarks for step-level evaluation in computer use. To bridge these gaps, we introduce OS-Oracle that makes three core contributions: (1) a scalable data pipeline for synthesizing cross-platform GUI critic data; (2) a two-stage training paradigm combining supervised fine-tuning (SFT) and consistency-preserving group relative policy optimization (CP-GRPO); (3) OS-Critic Bench, a holistic benchmark for evaluating critic model performance across Mobile, Web, and Desktop platforms. Leveraging this framework, we curate a high-quality dataset containing 310k critic samples. The resulting critic model, OS-Oracle-7B, achieves state-of-the-art performance among open-source VLMs on OS-Critic Bench, and surpasses proprietary models on the mobile domain. Furthermore, when serving as a pre-critic, OS-Oracle-7B improves the performance of native GUI agents such as UI-TARS-1.5-7B in OSWorld and AndroidWorld environments. The code is open-sourced at <https://github.com/numbmelon/OS-Oracle>.

1. Introduction

Recent advances in Vision-Language Models (VLMs) [3, 6, 14, 28, 37, 40] not only demonstrate remarkable performance on general vision-language tasks, but also exhibit promising potential in understanding, navigating, and ma-

nipulating graphical user interfaces (GUIs). As such, VLM-based computer-using agents (CUAs) [9, 15, 22, 26, 29, 39, 46, 51] have attracted substantial attention in community. By interacting with GUI environments, these agents are capable of completing user-specified tasks autonomously, laying a solid foundation for digital task automation.

Despite rapid progress, current CUAs still expose several recurring limitations in practice: 1) *Operation Failure* (OF), which reflects a failure to perceive subtle state transitions, leading to mistimed `type` before `click`, redundant post-click retries, or futile boundary scrolling; 2) *Inefficient Error State Recovery* (IESR), which arises when agents enter unexpected UI states yet fail to issue corrective operations such as `back`, allowing errors to compound; 3) *Mistimed Task Termination* (MTT), which denotes an imprecise understanding of termination states, causing premature stopping on unfinished tasks or unnecessary steps after successful completion; And 4) *Inaccurate Element Localization* (IEL), which often occurs when the wrong UI elements are selected or when the click position falls outside the activation area of the UI element. The first three issues primarily stem from weaknesses in planning and step-level reasoning under partial observability and long horizons, whereas IEL reflects limited UI element perception and grounding. These deficiencies degrade robustness across desktop, mobile, and web environments and hinder reliable long-horizon task completion.

To mitigate these limitations, existing efforts follow two main routes. The first line optimizes native agents with post-training optimization based on offline or online reinforcement learning (RL) [13, 18, 29, 38, 53], which can improve planning, error recovery, and termination decisions, and sometimes alleviate localization errors for models. Yet it is difficult to optimize, sensitive to reward design, and costly due to large-scale environment interaction, particularly for complex tasks that involve long-horizon planning and reasoning. The alternative route focuses on the Critic Model [36, 43, 47], which utilizes a verifier to assess the correctness of each step when agents interact with environments, offering a more cost-effective and scalable solution.

* Equal Contribution.

† Corresponding Author.

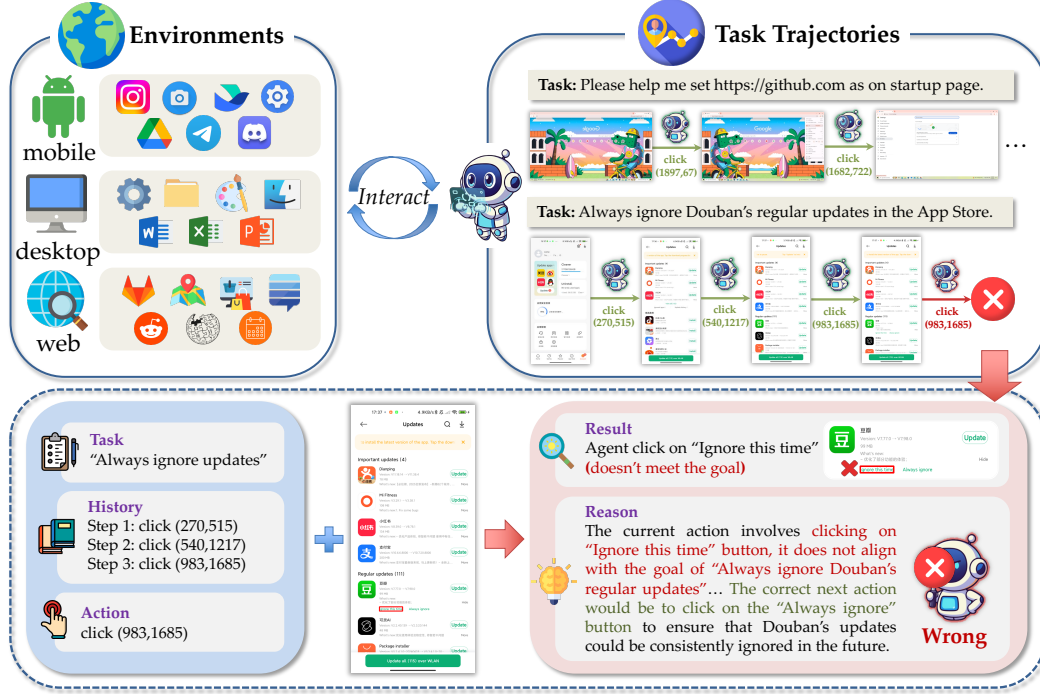


Figure 1. An example of CUA automation with a step-wise critic model. The critic model analyzes the current task, interaction history, the CUA’s proposed next action, and the current screenshot, and then judges whether the action is correct.

Critic models can be trained on smaller VLMs and then applied to various agents without extensive retraining. However, despite their potential, Critic models for computer use are still underdeveloped due to the lack of i) *open-source multi-platform critic data for training* and ii) *a well-defined benchmark for evaluation*. As a result, there remains significant room to advance the development of Critic Models.

To tackle these gaps, we present **OS-Oracle**, a full-stack framework that integrates data curation, training recipes, and evaluation protocols for developing critic models. **First**, we build an automatic critic data pipeline that derives hard negatives from existing positive trajectories to target the four error types, *i.e.*, OF, IESR, MTT, and IEL. We begin by extracting positive triplets $\langle \text{instruction}, \text{screenshot}, \text{action} \rangle$ from trajectory datasets. We then apply a rule-based policy to convert the gold action into a type-specific erroneous action. For each negative sample, we inject the explicit error tag into the prompt and use GPT-4o to annotate concise rationales, yielding a high-quality corpus of $\sim 160k$ positives and $\sim 150k$ negatives. **Second**, based on Qwen2.5VL-7B [6], we train a visual critic model termed as *OS-Oracle-7B* with a two-stage recipe. Stage one uses *supervised fine-tuning* to establish core discrimination and rationale skills. Stage two applies *Consistency-Preserving Group Relative Policy Optimization* (CP-GRPO), which utilizes a consistency reward to align the model’s reasoning content with its fi-

nal judgment, improving both discriminability and reasoning-judgment agreement. **Third**, we construct OS-Critic Bench, the first cross-platform GUI critic benchmark for mobile, web, and desktop. We sample trajectories from existing computer-using benchmarks, and generate one action proposal per screenshot with Qwen2.5VL-7B due to its balanced accuracy in practice. Then, human experts are engaged to check and assign a binary label to indicate correctness, mitigating noise cases that can arise when multiple feasible actions exist. We finally obtain a reliably curated set of 738 instances for this benchmark. Generally, our OS-Oracle offers an end-to-end pathway to curate targeted data, train consistent and scalable critics, and evaluate them rigorously, with the goal of improving task success rates for computer-using agents in real environments.

We summarize contributions as follows:

- (1) We propose a scalable data pipeline for synthesizing cross-platform GUI critic data, generating a dataset with $\sim 160k$ positive and $\sim 150k$ negative samples. It enables training critic models across diverse platforms.
- (2) We introduce an elaborate training recipe that integrates supervised fine-tuning with consistency-preserving group relative policy optimization. Training with this strategy, our critic model, *i.e.*, OS-Oracle-7B, exhibits strong consistency on reasoning and judgment.
- (3) We present OS-Critic Bench, a comprehensive benchmark designed for evaluating critic model perfor-

mance across mobile, web, and desktop platforms.

(4) The experiments demonstrate that our model sets new state-of-the-art results among open-source VLMs. When integrated with UI-TARS-1.5-7B, our critic model yields consistent performance gains across multiple online benchmarks (*e.g.*, 29.2% to 31.0% on OSWorld).

2. Related Work

Computer-Using Agents. Building on the rapid progress of general-purpose Vision-Language Models (VLMs) [4, 6, 10, 16, 40], computer-using agents (CUAs) have made significant advances in interacting with operating system environments, which enable VLMs serving as copilots to complete digital tasks [2, 9, 27, 33, 49, 55]. Native agents [22, 29] integrate planning and grounding within a single, end-to-end model. These agents interact directly with the environment based on user instructions with raw visual observations [38, 41, 46, 50]. As an alternative paradigm, modular agent frameworks [1, 12, 32, 45, 49, 52] consist of several sub-agents for planning, reflection, and memory, respectively, which exhibit promising computer-using performance. Despite that, current CUAs [22, 26, 29, 38, 41, 53] still frequently generate infeasible or inaccurate actions that lead to task failure, underscoring an urgent need for a step-level critic model tailored to CUAs.

Critic Models for CUAs. Critic models play an important role in CUAs across data collection, model training, and online task completion [13, 21, 24, 36, 38, 41, 53]. Existing critics for CUAs fall into two families. Broadly, existing work instantiates critics in two forms: general-purpose VLMs used as in-context critics, and instruction-tuned VLMs specialized for computer-using. On the one hand, OS-Genesis [33] and NNetaNav [25] prompt off-the-shelf VLMs to score raw trajectories, yielding higher-quality training data, and some systems [44, 51] adopt general-purpose VLMs as external reflection modules during online interaction. However, such in-context critics based on general-purpose VLMs exhibit strong acceptance bias and can produce unstable judgments [21, 24]. On the other hand, a line of work trains VLMs as reward models tailored for CUAs [29, 34, 36, 43, 47]. For instance, UI-TARS-2 [38] adopt the model itself as an outcome reward model (ORM) to validate web tasks in RL, UI-Critic-R1 [43] and UI-Genie [47] further provide step-wise feedback at inference time, but are trained solely on mobile data, limiting cross-platform generalization. In contrast, we train a unified critic model that delivers step-level, reasoning-grounded judgments across mobile, web, and desktop, to support the full CUA lifecycle.

Computer-Using Benchmarks. A variety of benchmarks have emerged to assess computer-using abilities on several aspects, including GUI content understanding [42], GUI grounding [9, 19, 42, 46, 49], and GUI task comple-

tion [7, 8, 11, 17, 20, 23, 30, 31, 35, 42, 48, 56]. In addition, several benchmarks for critic evaluation of computer use agents have been proposed. GUI-Critic-Test [43] is built from open-source data and is thus vulnerable to data leakage, while AgentRewardBench [24] and CUARewardBench [27] are manually constructed but restricted to web and desktop settings, respectively. In contrast, our OS-Critic Bench is curated from CUAs’ inference trajectories with human annotation, covering mobile, web, and desktop domains. We believe it provides a comprehensive and precise evaluation of models’ critic capabilities for CUAs.

3. OS-Oracle Data Pipeline

3.1. Background

CUAs Formulation. The process of GUI automation task can be formalized as a Partially Observable Markov Decision Process (POMDP) $\langle g, \mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T} \rangle$. Here, g denotes the task goal, $\mathcal{S}$ the state space, $\mathcal{A}$ the action space, $\mathcal{O}$ the observation space, and $\mathcal{T} : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ the state transition function. At each time step t , the agent executes a decision based on policy π , which integrates the task goal g , memory $m_t = \{o_j, a_j, o_{j+1}, a_{j+1}, \dots, o_{t-1}, a_{t-1}\}, 0 \leq j < t$, and the current observation o_t . The agent’s trajectory, $\tau = [s_0, a_0, s_1, a_1, \dots, s_t]$, emerges from the policy and state transitions, as formulated by:

$$p_\pi(\tau) = p(s_0) \prod_{t=0}^T \pi(a_t | g, s_t, m_t) \mathcal{T}(s_{t+1} | s_t, a_t). \quad (1)$$

Critic Models. Approaches for critic models generally fall into two categories: (i) Pointwise scoring, where the model evaluates each candidate response independently; and (ii) Pairwise ranking, which identifies the better response from a given pair of candidates. We adopted the pointwise scoring scheme because a pairwise critic model is ill-suited for GUI tasks, as it is forced to determine a preference between two actions, even if both are functionally ineffective. The pointwise scoring, in contrast, is trained to evaluate each action independently against an objective standard of correctness, providing a clear and interpretable score of its validity for task completion. A pointwise critic model $\mathcal{M}_{\text{critic}}$ for CUAs is formulated to assess the quality or correctness of an agent’s action $a_t \in \mathcal{A}$ at a given time step t :

$$r_t, j_t = \mathcal{M}_{\text{critic}}(g, m_t, o_t, a_t), \quad (2)$$

where r_t represents the reason for action assessment and j_t represents the judgment, which can be a continuous value in $[0, 1]$ or a binary value of 0 or 1. Our goal is to determine step-level validity. Therefore, we define the critic’s output as a binary value $j_i \in \{\text{Yes}, \text{No}\}$, where the former signifies a correct action and No signifies an incorrect one.

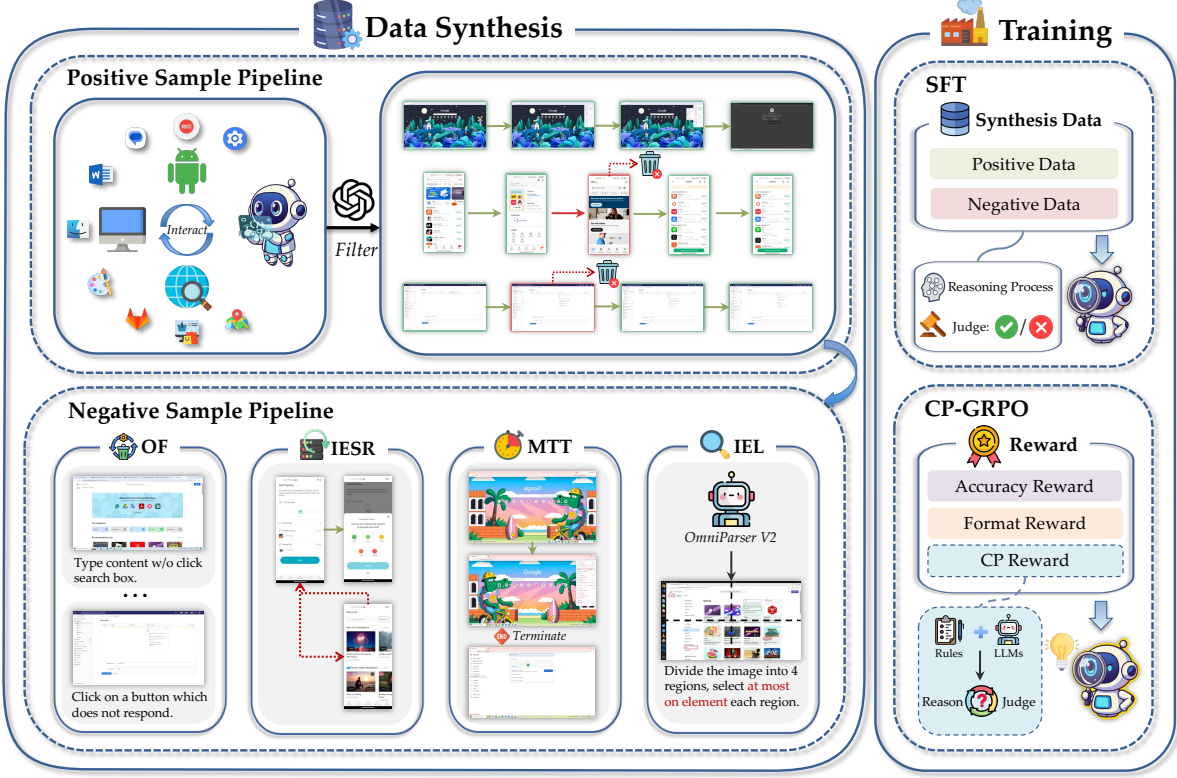


Figure 2. The framework of OS-Oracle. (Left) Data Synthesis. We first use GPT-based filtering to extract step-wise positive samples from raw trajectories, and then construct four types of negative samples from the positive ones: Operation Failure (OF), Inefficient Error State Recovery (IESR), Mistimed Task Termination (MTT), and Inaccurate Element Localization (IEL). (Right) Two-stage training: supervised fine-tuning (SFT) on synthesized data followed by CP-GRPO training with accuracy, format, and consistency-preserving rewards.

3.2. Data Construction

To successfully transform VLMs into binary pointwise critic models, a central bottleneck is the extreme scarcity of requisite training data, particularly high-quality negative samples. Publicly available datasets for GUI planning are already scarce, and the vast majority consist of expert demonstrations composed almost entirely of correct operations, leading to an inherent lack of negative samples. Moreover, distinguishing between correct and erroneous operations within these trajectories generated by CUAs is exceptionally challenging. Even when a CUA successfully completes a task, its execution path may contain numerous redundant, inefficient, or unnecessary steps. Conversely, for trajectories where the CUA fails, precisely identifying the specific erroneous step responsible for the failure is notoriously difficult in practice. Consequently, a straightforward strategy based solely on the final task outcome to determine correctness of steps is highly unreliable. Compounding this issue, the lack of robust GUI planning knowledge in current open-source VLMs makes correctness judgement difficult and further exacerbates negative sample collection. These factors collectively impede the development of GUI critic

models capable of effectively identifying operational errors.

To address the aforementioned bottleneck, we have meticulously designed an automated negative sample construction pipeline as shown in Figure 2, which bypass the difficult requirement of explicitly judging the correctness of each step within a trajectory. We analyze the key error patterns of CUAs and simulate these typical failure scenarios to automatically construct high-fidelity negative samples from existing trajectory datasets. Specifically, our pipeline targets several critical deficiencies observed in CUAs, and constructs corresponding erroneous interaction contexts based on existing trajectory datasets to help the critic effectively distinguish these errors.

Operation Failure (OF). It addresses the lack of perception for subtle state changes by simulating three key error scenarios: (1) synthetically inserting ‘type’ actions before their corresponding ‘click’ actions, to simulate the failure to perceive the subtle visual activation of an input field; (2) artificially repeating eligible click operations, to simulate the failure to register minimal, localized UI changes post-click; and (3) appending redundant ‘scroll up’ or ‘scroll down’ actions, to simulate the failure to perceive the absence of

visual change when at a scroll boundary.

Inefficient Error State Recovery (IESR). The pipeline targets the lack of recovery capabilities from error states. Agents often fail to execute a ‘back’ operation or other corrective measures when encountering an unexpected UI state. We address this using a state-injection method. For a given correct step, we first use its observation to retrieve trajectories with highly similar observations and then randomly select one of these trajectories to extract its subsequent step injected into the given step as a synthetic error next state, where we posit that all operations except for the ‘back’ action should be considered negative samples.

Mistimed Task Termination (MTT). We address insufficient understanding of task termination states, where agents may not recognize when a task is completed. We construct negatives by either appending redundant operations to a successful trajectory or by prematurely truncating an unfinished trajectory and appending a terminate action.

Inaccurate Element Localization (IEL). CUAs may choose the correct action type but apply it to the wrong UI element. To simulate this failure mode, we first use Omni-Parser V2 [54] to detect interactive elements. When metadata such as the AndroidControl accessibility tree is available, we extract element information from it and compute Intersection over Union (IoU) between metadata-defined and detected elements, retaining detected elements with $\text{IoU} > 0.7$. The remaining detected UI elements are then partitioned into a 2x2 grid over the image, and we sample at most one element per cell to obtain diverse error candidates.

Through the four error patterns corresponding data construction, we obtained erroneous operations as negative samples. With operations from existing trajectories as positive samples, we consequently derived a prompt set suitable for training critic models.

3.3. Data Annotation

Given an initial dataset of agent interaction contexts and corresponding judgements, we employ GPT-4o [16] to generate reasons for these judgements. This crucial step, termed rationale generation, effectively bootstraps the originally sparse labels into an information-rich set of explanatory rationales. Training on this enriched data enables the model to internalize the evaluation process rather than mimic final outcomes, resulting in more robust alignment.

Specifically, since negative samples are constructed for specific failure modes, we provide GPT-4o with the task history, incorrect actions, and the corresponding error category and cause. This signal helps GPT-4o generate more precise rationales and better align its criticism with the task context, thereby improving critic capacity. Prompt templates are given in the Appendix. We discard samples where GPT-4o’s judgment of operational correctness conflicts with the ground-truth labels to ensure annotation quality. Using this

pipeline, we construct the OS-Oracle training set with 160k positive and 150k negative samples.

4. OS-Oracle Training

Based on our critic datasets, we propose a two-stage training strategy to effectively train the OS-Oracle critic model, combining supervised fine-tuning and a novel reinforcement learning objective.

4.1. Supervised Fine-tuning

The training pipeline begins with Supervised Fine-Tuning (SFT), which serves as the foundational stage for initializing the model with task-specific knowledge. Starting from a VLM, we fine-tune the model parameters θ on the OS-Oracle training dataset $\mathcal{D} = \{(x_i, r_i, j_i)\}$, where each input $x_i = (g, m_t, o_t, a_t)$ represents a GUI context. The objective is to minimize the cross-entropy loss for both the reason generation and judgment prediction:

$$\mathcal{L}_{\text{SFT}} = -\mathbb{E}_{(x,r,j) \sim \mathcal{D}} [\log P_{\theta}(r | x) + \log P_{\theta}(j | x, r)]. \quad (3)$$

This stage ensures that the model acquires a robust initial ability to generate plausible reasoning traces and correct judgments before advancing to reinforcement learning.

4.2. Consistency-Preserving GRPO

To further enhance the discriminative capability and alignment of the model, we proceed to the second stage where this stage refines the SFT model $\pi_{\theta}^{\text{SFT}}$ into an improved policy π_{θ} through reinforcement learning. We build upon the Group Relative Policy Optimization (GRPO) framework, and employs a group-based approach: for a group of N semantically similar queries $\mathcal{G} = \{x_1, x_2, \dots, x_N\}$, we sample outputs $\{\hat{r}_i, \hat{j}_i\} \sim \pi_{\theta}(\cdot | x_i)$ and compute rewards that facilitate relative comparisons within the group. The optimization objective is to maximize the expected reward:

$$\mathcal{J}_{\text{RL}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \left[\mathbb{E}_{\hat{r}, \hat{j} \sim \pi_{\theta}(\cdot | x)} \left[R(x, \hat{r}_i, \hat{j}_i) \right] \right], \quad (4)$$

where $R(x, \hat{r}_i, \hat{j}_i)$ denotes the reward function. During GRPO training, we observed inconsistency between generated reasons and final judgments: the model sometimes produces reasons that logically support a correct action, yet the final judgment incorrectly classifies the action as wrong.

To address this issue, we introduce Consistency-Preserving GRPO (CP-GRPO) incorporating a consistency reward $R_{\text{consistency}}$ via a hybrid rule-based and model-based scheme, which explicitly encourages alignment between the reasoning content and the judgment outcome.

Specially, we first employ a rule-based method to determine the polarity of a predicted reason $\hat{r}_i$ based on a positive lexicon $\mathcal{L}^+$, indicating the action is helpful for completing

the task and a negative lexicon $\mathcal{L}^-$ which represents the action cannot help complete the task. The reasons is extracted and segment it into semantically continuous units. Then, we respectively count the number of occurrences of these units from the positive and negative lexicons:

$$c_i^+ = \sum_{w \in \mathcal{T}(\hat{r}_i)} \mathbf{1}[w \in \mathcal{L}^+], \quad c_i^- = \sum_{w \in \mathcal{T}(\hat{r}_i)} \mathbf{1}[w \in \mathcal{L}^-], \quad (5)$$

where $\mathbf{1}$ is the indicator function and $\mathcal{T}$ represents split function. The resulting score is $s_i^{\text{rule}} = c_i^+ - c_i^-$, which determines the rule-based polarity p_i^{rule} based on $p_i^{\text{rule}} = \text{sgn}(s_i^{\text{rule}})$, where $\text{sgn}(0) = \emptyset$.

If $p_i^{\text{rule}} = \emptyset$, we fall back to a model-based approach, where we feed Qwen3-8B with the reason to directly generate a proxy judgment and polarity p_i^{model} is determined by comparing the logits:

$$p_i^{\text{model}} = \begin{cases} +1, & \ell_i^{\text{Yes}} > \ell_i^{\text{No}}, \\ -1, & \ell_i^{\text{Yes}} \leq \ell_i^{\text{No}}, \end{cases} \quad (6)$$

where ℓ_i^{Yes} and ℓ_i^{No} denote the logits for Yes and No, respectively. The final polarity for the reason is calculated as:

$$\tilde{p}_i = \begin{cases} p_i^{\text{rule}}, & \text{if } p_i^{\text{rule}} \neq \emptyset, \\ p_i^{\text{model}}, & \text{if } p_i^{\text{rule}} = \emptyset. \end{cases} \quad (7)$$

We define the consistency reward $R_{\text{consistency}}$ as 1 if $\tilde{p}_i$ matches judgment j_i , and 0 otherwise, as follows:

$$R_{\text{consistency}}(x_i, \hat{r}_i, \hat{j}_i) = \mathbf{1}[\tilde{p}_i = \nu(\hat{j}_i)], \quad (8)$$

where $\nu(\text{Yes})=+1$ and $\nu(\text{No})=-1$. The overall CP-GRPO reward can be formulated:

$$R(x_i, \hat{r}_i, \hat{j}_i) = \alpha R_{\text{acc}} + \beta R_{\text{format}} + \gamma R_{\text{consistency}}, \quad (9)$$

where the accuracy reward R_{acc} is defined as $\mathbf{1}[\hat{j}_i = j_i]$ and format reward R_{format} is a binary score indicating whether the generated reason $\hat{j}_i$ conforms to the required structure.

5. Experiments

5.1. OS-Critic Bench

To evaluate the versatility and generalization of critic models. We introduce **OS-Critic Bench**, the first cross-platform benchmark underwent rigorous quality control.

Data Source. To ensure comprehensive cross-platform evaluation, the benchmark construction utilizes data from three distinct domains. The Mobile component is sourced from the AndroidControl [20] test set and the GUIOdyssey [23] test set. For the Web platform, we incorporate data from the guiaact [8] test set, supplemented by a non-overlapping subset of ScaleCUA-Web [22] that is

reserved exclusively for benchmarking. Finally, Desktop scenarios are derived from AgentNet-Bench [41].

Operation Sampling. We utilize these task prompts and corresponding screenshots to sample candidate operations using the Qwen2.5-VL-7B-Instruct model, yielding a large amount of data available for evaluation by the critic model.

Quality Control. Crucially, due to the inherent non-unique nature of solutions in complex GUI-based tasks, we departed from traditional rule-based methods that rely strictly on comparing generated actions against existing ground truth actions for judgment. Instead, we introduced human experts was tasked with providing the final and definitive judgment for the correctness of the sampled actions, even if the action deviated from the original trajectory ground truth but still constituted a valid step towards the task goal. This meticulous, human-centric validation process ensured the high quality and fidelity of the resulting dataset, yielding a substantial volume of reliable data available for robust evaluation by the critic model.

Through this process, we collected 738 samples for evaluation. Each instance in the dataset comprises a task goal, memory, a screenshot, an action under review, a concise prompt serving as critic, and a human-annotated binary label indicating whether the action advances task completion.

5.2. Setup

Training. We adopt Qwen2.5-VL-7B-Instruct as the backbone and train it in two phases. During the SFT phase, we train on the entire OS-Oracle dataset with 1 epoch, resulting in OS-Oracle-7B-SFT. Subsequently, in the CP-GRPO phase, we further optimize OS-Oracle-7B-SFT on the same dataset for 3 epochs to obtain OS-Oracle-7B, where the KL regularization coefficient is set to 5×10^{-3} with reward weights $\alpha = 0.9$, $\beta = 0.05$, and $\gamma = 0.05$ in Eq. (9). Rollout batch size and group size are 512 and 16, respectively.

Evaluation. We comprehensively evaluate the capabilities of OS-Oracle-7B across three aspects: (1) **Offline Evaluation:** Tests its judgment capability to compare agent’s prediction with predefined actions. (2) **Dynamic Evaluation:** Assesses its online assistance capability by measuring its ability to guide an agent toward task completion, showing its practical utility. (3) **Data Quality Control:** Probes its potential for automated data quality control on GUI datasets. More details are shown in Appendix.

5.3. Main results

Offline Evaluation. We evaluate a diverse set of general-purpose VLMs and CUAs on the OS-Critic Bench shown in Table 1. Proprietary VLMs, particularly GPT-5 and Claude-4.5-Sonnet, produce impressive results on Desktop while there is still potential for enhancement with regards to Web and Mobile. Current open-source VLMs, such as Qwen2.5-VL-7B and Qwen3-VL-8B, exhibit critic capabilities thanks

Table 1. Performance comparison on OS-Critic Bench. The table presents proprietary models, open-source models, and OS-Oracle. The **best** and second-best results are indicated in bold and underlined, respectively.

Model	Mobile		Desktop		Web		Overall	
	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score
Proprietary Models								
GPT-4o [16]	62.79	66.53	63.82	70.90	58.78	<u>65.92</u>	62.20	67.37
GPT-5 [28]	69.18	<u>69.53</u>	<u>73.68</u>	<u>74.36</u>	59.46	55.22	68.16	<u>67.94</u>
Claude-4.5-Sonnet [3]	65.30	63.46	74.34	77.46	<u>64.19</u>	64.90	66.94	67.03
Gemini-2.5-Pro [10]	<u>68.26</u>	70.11	67.11	70.24	66.22	70.24	<u>67.62</u>	70.16
Open Source Models								
Qwen2.5-VL-7B [6]	58.45	65.79	61.18	69.43	54.73	64.17	58.27	66.23
Qwen3-VL-8B [5]	61.64	62.33	59.87	63.91	60.14	60.40	60.98	62.30
UI-TARS-1.5-7B [29]	56.16	66.43	58.55	69.57	60.14	70.05	57.45	67.83
ScaleCUA-7B [22]	50.23	59.93	44.74	58.82	34.46	49.21	45.93	57.51
ScaleCUA-32B [22]	51.60	63.32	49.34	64.84	50.68	65.40	50.95	64.09
SE-WSM-7B [36]	50.91	16.99	50.66	13.79	41.22	18.69	48.92	16.78
GUI-Critic-R1 [43]	59.13	66.54	59.21	70.19	<u>60.81</u>	72.90	59.49	68.76
OS-Oracle-7B-SFT	<u>63.47</u>	<u>71.22</u>	<u>63.16</u>	72.00	62.16	<u>71.72</u>	<u>63.14</u>	<u>71.49</u>
OS-Oracle-7B	70.78	74.30	65.79	<u>71.11</u>	62.16	70.53	68.02	72.81

to their inherent reasoning ability, enabling them to assess GUI actions to some extent even without task-specific training. In contrast, existing CUAs such as ScaleCUA-7B and ScaleCUA-32B, while proficient in performing GUI operational tasks, are inherently less capable of evaluating the validity of actions, leading to noticeably lower critic performance compared to general-purpose VLMs. SE-WSM-7B, a world state model for CUAs, exhibits poor performance and generalization because it is trained on a dataset with limited diversity. Similarly, GUI-Critic-R1, although specifically designed as a critic model for GUI scenarios, shows only marginal improvement over its base model Qwen2.5-VL-7B due to the constrained domain coverage and limited scale of its training data. Our OS-Oracle-7B achieves the highest accuracy among all open-source models and delivers state-of-the-art performance on Mobile, highlighting the importance of extensive training for GUI critic models and the effectiveness of our large-scale, scalable data pipeline.

Dynamic Evaluation. We compared GPT-4o and OS-Oracle-7B as critic models to guide UI-TARS-1.5-7B in completing tasks within AndroidWorld and OSWorld. Specifically, if the critic model deemed an action incorrect, the CUAs were required to regenerate the action with a maximum of three attempts. The evaluation results are shown in Fig. 3, which demonstrate that across both platforms, the performance of the base agent is enhanced by OS-Oracle-7B, while GPT-4o tends to degrade the performance of CUAs due to a lack of relevant knowledge, result-

ing in inaccurate evaluations of operational elements and consequently leading the model into erroneous states.

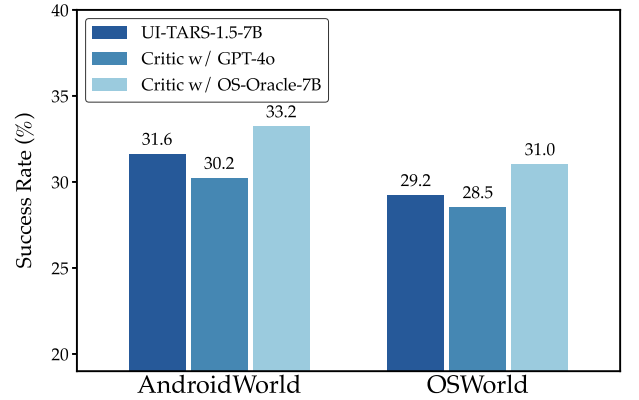


Figure 3. The performance of UI-TARS-1.5-7B with critic model on AndroidWorld and OSWorld.

Data Quality Control. We assessed the potential of the critic model to establish a data flywheel, which enables the collection of high-quality GUI trajectory data. Specifically, we initially employed GPT-4o to generate diverse tasks. These tasks served as instructions for two open-source CUA models (ScaleCUA-7B and UI-TARS-1.5-7B) to produce trajectories, yielding a total of 10K mobile operational samples. These samples are then used to train Qwen2.5-VL-7B-Instruct under two settings: (1) standard SFT on the entire 10k samples; (2) critic-guided SFT using OS-Oracle-

Table 2. Comparison of SFT performance on all rollout trajectories vs trajectories filtered by OS-Oracle-7B.

Model	AndroidWorld
Qwen2.5-VL-7B	10.34
+ SFT	12.07
+ SFT w/ OS-Oracle-7B	15.52

Table 3. Effect of Synthetic vs. GPT-Annotated Negative Samples.

Model	Overall	
	Acc	F1
Qwen2.5-VL-7B	58.27	66.23
+ w/ GPT-Annotated Negatives	55.42	67.33
+ w/ OS-Oracle Synthetic Negatives	60.03	69.37

7B, which utilizes OS-Oracle-7B to judge the correctness of actions within each sample, and then performs SFT on the curated subset of trajectories assessed by the critic as being valuable for task progression. To balance total training exposure, we train the baseline for 1 epoch on the full dataset and the critic-guided model for 2 epochs on the filtered subset. As shown in Table 2, the critic-guided SFT improves the downstream AndroidWorld performance by 3.45% over the SFT baseline. This demonstrates that OS-Oracle-7B can reliably filter low-quality actions, making it suitable for improving the quality of collected trajectories.

5.4. Ablation

The effect of synthetic negative data. We evaluate the quality of our synthetic incorrect operational samples by comparing with negative samples identified by GPT-4o, commonly adopted in prior studies [36, 43]. We randomly select 50k positive samples from the full dataset for both settings and pair them with either 50k GPT-annotated negatives or 50k OS-Oracle synthetic negatives to ensure fair comparison. The two SFT models, each trained on its corresponding dataset, are evaluated using the OS-Critic bench shown in Table 3. The model trained with our synthetic negative samples clearly outperforms the one trained with GPT-annotated negatives. Moreover, the GPT-annotated negative group performs worse than the baseline, suggesting that such naturally annotated negatives contain substantial noise rather than meaningful negative supervision.

The effect of the amount of critic data. To examine how the amount of critic data affects model performance, we perform SFT on four training sets of different scales, all based on the same initial model. Specifically, we used $\pm 10k$, $\pm 50k$, $\pm 100k$ positive and negative samples, respectively, as well as the full training set, to train the critic models and reported their performance on the OS-Critic Bench.

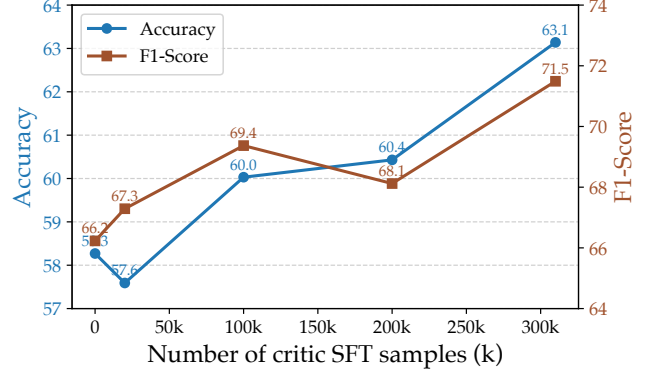


Figure 4. Effect of SFT data scaling on overall performance.

Table 4. Effect of consistency reward in CP-GRPO.

Method	Overall	
	Acc	Consistency
SFT + GRPO	66.53	80.89
SFT + CP-GRPO	68.02	99.73

The effect of consistency reward. To assess the impact of incorporating a consistency reward into the GRPO objective, we utilize OS-Oracle-7B-SFT as the initial model and then perform both GRPO and CP-GRPO training on the full dataset to compare the accuracy and the consistency calculated based on Eq (8). As detailed in Table 4, both methods achieve comparable accuracy while the consistency of GRPO is approximately 20% lower than that of CP-GRPO. This disparity indicates that standard GRPO tends to induce mismatches between reasoning and judgment during GUI critic model training, whereas our CP-GRPO effectively mitigates these inconsistencies.

6. Conclusion

In this work, we present **OS-Oracle**, a comprehensive framework for GUI critic models. By introducing a scalable cross-platform data pipeline, we systematically synthesize both positive and negative samples that capture diverse GUI failure modes. Together with a two-stage training recipe combining supervised fine-tuning and consistency-preserving GRPO, our approach enables robust and generalizable critic learning across Mobile, Web, and Desktop environments. Extensive experiments demonstrate that our critic model not only achieves impressive performance on the OS-Critic Bench but also effectively enhances the reliability and task success of native GUI agents. We hope this work will inspire future research and development of more capable and reliable GUI critic models.

Acknowledgments

This work is supported by the Shanghai Artificial Intelligence Laboratory. The authors gratefully acknowledge its support.

References

- [1] Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent s: An open agentic framework that uses computers like a human. *arXiv preprint arXiv:2410.08164*, 2024. 3
- [2] Anthropic. Developing a computer use model, 2024. Product announcement. 3
- [3] Anthropic. Introducing Claude Sonnet 4.5. <https://www.anthropic.com/news/claude-sonnet-4-5>, 2025. Published: 2025-09-29; Accessed: 2025-10-27. 1, 7
- [4] Sonnet Anthropic. Claude 3.7 sonnet system card, 2025. 3
- [5] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibao Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025. 7
- [6] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025. 1, 2, 3, 7
- [7] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Buckner, et al. Windows agent arena: Evaluating multi-modal os agents at scale. *arXiv preprint arXiv:2409.08264*, 2024. 3
- [8] Wentong Chen, Junbo Cui, Jinyi Hu, Yujia Qin, Junjie Fang, Yue Zhao, Chongyi Wang, Jun Liu, Guirong Chen, Yupeng Huo, Yuan Yao, Yankai Lin, Zhiyuan Liu, and Maosong Sun. Guicourse: From general vision language models to versatile gui agents, 2024. 3, 6
- [9] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9313–9332, Bangkok, Thailand, 2024. Association for Computational Linguistics. 1, 3
- [10] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blisstein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025. 3, 7
- [11] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114, 2023. 3
- [12] Gonzalo Gonzalez-Pumariiega, Vincent Tu, Chih-Lun Lee, Jiachen Yang, Ang Li, and Xin Eric Wang. The unreasonable effectiveness of scaling agents for computer use. *arXiv preprint arXiv:2510.02250*, 2025. 3
- [13] Zhangxuan Gu, Zhengwen Zeng, Zhenyu Xu, Xingran Zhou, Shuheng Shen, Yunfei Liu, Beitong Zhou, Changhua Meng, Tianyu Xia, Weizhi Chen, et al. Ui-venus technical report: Building high-performance ui agents with rft. *arXiv preprint arXiv:2508.10833*, 2025. 1, 3
- [14] Dong Guo, Faming Wu, Feida Zhu, Fuxing Leng, Guang Shi, Haobin Chen, Haoqi Fan, Jian Wang, Jianyu Jiang, Jiawei Wang, et al. Seed1. 5-vl technical report. *arXiv preprint arXiv:2505.07062*, 2025. 1
- [15] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14281–14290, 2024. 1
- [16] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024. 3, 5, 7
- [17] Raghav Kapoor, Yash Parag Butala, Melisa Russak, Jing Yu Koh, Kiran Kamble, Waseem Alshikh, and Ruslan Salakhutdinov. Omniaact: A dataset and benchmark for enabling multimodal generalist autonomous agents for desktop and web, 2024. 3
- [18] Hanyu Lai, Xiao Liu, Yanxiao Zhao, Han Xu, Hanchen Zhang, Bohao Jing, Yanyu Ren, Shuntian Yao, Yuxiao Dong, and Jie Tang. Computerrl: Scaling end-to-end online reinforcement learning for computer use agents. *arXiv preprint arXiv:2508.14040*, 2025. 1
- [19] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use. *arXiv preprint arXiv:2504.07981*, 2025. 3
- [20] Wei Li, William Bishop, Alice Li, Chris Rawles, Folawiyi Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on computer control agents, 2024. 3, 6
- [21] Haojia Lin, Xiaoyu Tan, Yulei Qin, Zihan Xu, Yuchen Shi, Zongyi Li, Gang Li, Shaofei Cai, Siqi Cai, Chaoyou Fu, et al. Cuarewardbench: A benchmark for evaluating reward models on computer-using agent. *arXiv preprint arXiv:2510.18596*, 2025. 3
- [22] Zhaoyang Liu, Jingjing Xie, Zichen Ding, Zehao Li, Bowen Yang, Zhenyu Wu, Xuehui Wang, Qiushi Sun, Shi Liu,

- Weiyun Wang, Shenglong Ye, Qingyun Li, Xuan Dong, Yue Yu, Chenyu Lu, YunXiang Mo, Yao Yan, Zeyue Tian, Xiao Zhang, Yuan Huang, Yiqian Liu, Weijie Su, Gen Luo, Xianguyu Yue, Biqing Qi, Kai Chen, Bowen Zhou, Yu Qiao, Qifeng Chen, and Wenhai Wang. Scalecua: Scaling open-source computer use agents with cross-platform data. *arXiv preprint arXiv:2509.15221*, 2025. Preprint. 1, 3, 6, 7
- [23] Quanfeng Lu, Wenqi Shao, Zitao Liu, Fanqing Meng, Boxuan Li, Botong Chen, Siyuan Huang, Kaipeng Zhang, Yu Qiao, and Ping Luo. Gui odyssey: A comprehensive dataset for cross-app gui navigation on mobile devices. *arXiv preprint arXiv:2406.08451*, 2024. 3, 6
- [24] Tianyi Men, Zhuoran Jin, Pengfei Cao, Yubo Chen, Kang Liu, and Jun Zhao. Agent-rewardbench: Towards a unified benchmark for reward modeling across perception, planning, and safety in real-world multimodal agents. *arXiv preprint arXiv:2506.21252*, 2025. 3
- [25] Shikhar Murty, Dzmitry Bahdanau, and Christopher D Manning. Nnatscape navigator: Complex demonstrations for web agents without a demonstrator. *arXiv e-prints*, pages arXiv–2410, 2024. 3
- [26] Dang Nguyen, Jian Chen, Yu Wang, Gang Wu, Namyong Park, Zhengmian Hu, Hanjia Lyu, Junda Wu, Ryan Aponte, Yu Xia, et al. Gui agents: A survey. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22522–22538, 2025. 1, 3
- [27] OpenAI. Computer-using agent: Introducing a universal interface for ai to interact with the digital world, 2025. 3
- [28] OpenAI. Introducing GPT-5. <https://openai.com/index/introducing-gpt-5>, 2025. Accessed: 2025-10-27. 1, 7
- [29] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025. 1, 3, 7
- [30] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Androidinthewild: A large-scale dataset for android device control. *Advances in Neural Information Processing Systems*, 36:59708–59728, 2023. 3
- [31] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. Androidworld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*, 2024. 3
- [32] Linxin Song, Yutong Dai, Viraj Prabhu, Jieyu Zhang, Taiwei Shi, Li Li, Junnan Li, Silvio Savarese, Zeyuan Chen, Jieyu Zhao, et al. Coact-1: Computer-using agents with coding as actions. *arXiv preprint arXiv:2508.03923*, 2025. 3
- [33] Qiushi Sun, Kanzhi Cheng, Zichen Ding, Chuanyang Jin, Yian Wang, Fangzhi Xu, Zhenyu Wu, Chengyou Jia, Liheng Chen, Zhoumianze Liu, et al. Os-genesis: Automating gui agent trajectory construction via reverse task synthesis. *arXiv preprint arXiv:2412.19723*, 2024. 3
- [34] Qiushi Sun, Mukai Li, Zhoumianze Liu, Zhihui Xie, Fangzhi Xu, Zhangyue Yin, Kanzhi Cheng, Zehao Li, Zichen Ding, Qi Liu, et al. Os-sentinel: Towards safety-enhanced mobile gui agents via hybrid validation in realistic workflows. *arXiv preprint arXiv:2510.24411*, 2025. 3
- [35] Qiushi Sun, Zhoumianze Liu, Chang Ma, Zichen Ding, Fangzhi Xu, Zhangyue Yin, Haiteng Zhao, Zhenyu Wu, Kanzhi Cheng, Zhaoyang Liu, et al. Scienceboard: Evaluating multimodal autonomous agents in realistic scientific workflows. *arXiv preprint arXiv:2505.19897*, 2025. 3
- [36] Zeyi Sun, Ziyu Liu, Yuhang Zang, Yuhang Cao, Xiaoyi Dong, Tong Wu, Dahua Lin, and Jiaqi Wang. Seagent: Self-evolving computer use agent with autonomous learning from experience. *arXiv preprint arXiv:2508.04700*, 2025. 1, 3, 7, 8
- [37] V Team, Wenyi Hong, Wenmeng Yu, Xiaotao Gu, Guo Wang, Guobing Gan, Haomiao Tang, Jiale Cheng, Ji Qi, Junhui Ji, Lihang Pan, Shuaiqi Duan, Weihang Wang, Yan Wang, Yean Cheng, Zehai He, Zhe Su, Zhen Yang, Ziyang Pan, Aohan Zeng, Baoxu Wang, Bin Chen, Boyan Shi, Changyu Pang, Chenhui Zhang, Da Yin, Fan Yang, Guoqing Chen, Jiazhen Xu, Jiale Zhu, Jiali Chen, Jing Chen, Jinhao Chen, Jinghao Lin, Jinjiang Wang, Junjie Chen, Leqi Lei, Letian Gong, Leyi Pan, Mingdao Liu, Mingde Xu, Mingzhi Zhang, Qinkai Zheng, Sheng Yang, Shi Zhong, Shiyu Huang, Shuyuan Zhao, Siyan Xue, Shangqin Tu, Shengbiao Meng, Tianshu Zhang, Tianwei Luo, Tianxiang Hao, Tianyu Tong, Wenkai Li, Wei Jia, Xiao Liu, Xiaohan Zhang, Xin Lyu, Xinyue Fan, Xuancheng Huang, Yanling Wang, Yadong Xue, Yanfeng Wang, Yanzi Wang, Yifan An, Yifan Du, Yiming Shi, Yiheng Huang, Yilin Niu, Yuan Wang, Yuanchang Yue, Yuchen Li, Yutao Zhang, Yuting Wang, Yu Wang, Yuxuan Zhang, Zhao Xue, Zhenyu Hou, Zhengxiao Du, Zihan Wang, Peng Zhang, Debing Liu, Bin Xu, Juanzi Li, Minlie Huang, Yuxiao Dong, and Jie Tang. Glm-4.5v and glm-4.1v-thinking: Towards versatile multimodal reasoning with scalable reinforcement learning, 2025. 1
- [38] Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Juntong Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, et al. Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544*, 2025. 1, 3
- [39] Shuai Wang, Weiwen Liu, Jingxuan Chen, Yuqi Zhou, Weinan Gan, Xingshan Zeng, Yuhang Che, Shuai Yu, Xinlong Hao, Kun Shao, et al. Gui agents with foundation models: A comprehensive survey. *arXiv preprint arXiv:2411.04890*, 2024. 1
- [40] Weiyun Wang, Zhangwei Gao, Lixin Gu, Hengjun Pu, Long Cui, Xingguang Wei, Zhaoyang Liu, Linglin Jing, Shenglong Ye, Jie Shao, et al. Internvl3. 5: Advancing open-source multimodal models in versatility, reasoning, and efficiency. *arXiv preprint arXiv:2508.18265*, 2025. 1, 3
- [41] Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, Zhennan Shen, Zhuokai Li, Ryan Li, Xiaochuan Li, Junda Chen, Boyuan Zheng, Peihang Li, Fangyu Lei, Ruisheng Cao, Yeqiao Fu, Dongchan Shin, Martin Shin, Jiarui Hu, Yuyan Wang, Jixuan Chen, Yuxiao Ye, Danyang Zhang, Dikang Du, Hao Hu, Huarong Chen, Zaida Zhou,

- Haotian Yao, Ziwei Chen, Qizheng Gu, Yipu Wang, Heng Wang, Diyi Yang, Victor Zhong, Flood Sung, Y. Charles, Zhilin Yang, and Tao Yu. Opencua: Open foundations for computer-use agents, 2025. 3, 6
- [42] Xuehui Wang, Zhenyu Wu, JingJing Xie, Zichen Ding, Bowen Yang, Zehao Li, Zhaoyang Liu, Qingyun Li, Xuan Dong, Zhe Chen, et al. Mmbench-gui: Hierarchical multi-platform evaluation framework for gui agents. *arXiv preprint arXiv:2507.19478*, 2025. 3
- [43] Yuyang Wanyan, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Jiabo Ye, Yutong Kou, Ming Yan, Fei Huang, Xiaoshan Yang, et al. Look before you leap: A gui-critic-r1 model for pre-operative error diagnosis in gui automation. *arXiv preprint arXiv:2506.04614*, 2025. 1, 3, 7, 8
- [44] Qianhui Wu, Kanzhi Cheng, Rui Yang, Chaoyun Zhang, Jianwei Yang, Huiqiang Jiang, Jian Mu, Baolin Peng, Bo Qiao, Reuben Tan, et al. Gui-actor: Coordinate-free visual grounding for gui agents. *arXiv preprint arXiv:2506.03143*, 2025. 3
- [45] Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. Os-copilot: Towards generalist computer agents with self-improvement. *arXiv preprint arXiv:2402.07456*, 2024. 3
- [46] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. Os-atlas: A foundation action model for generalist gui agents. *arXiv preprint arXiv:2410.23218*, 2024. 1, 3
- [47] Han Xiao, Guozhi Wang, Yuxiang Chai, Zimu Lu, Weifeng Lin, Hao He, Lue Fan, Liuyang Bian, Rui Hu, Liang Liu, et al. Ui-genie: A self-improving approach for iteratively boosting mllm-based mobile gui agents. *arXiv preprint arXiv:2505.21496*, 2025. 1, 3
- [48] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024. 3
- [49] Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, Wenjing Hu, Xinyuan Wang, Yuhui Xu, Zekun Wang, Yiheng Xu, Junli Wang, Doyen Sahoo, Tao Yu, and Caiming Xiong. Scaling computer-use grounding via user interface decomposition and synthesis, 2025. 3
- [50] Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguis: Unified pure vision agents for autonomous gui interaction. *arXiv preprint arXiv:2412.04454*, 2024. 3
- [51] Chenyu Yang, Shiqian Su, Shi Liu, Xuan Dong, Yue Yu, Weijie Su, Xuehui Wang, Zhaoyang Liu, Jinguo Zhu, Hao Li, et al. Zerogui: Automating online gui learning at zero human cost. *arXiv preprint arXiv:2505.23762*, 2025. 1, 3
- [52] Yan Yang, Dongxu Li, Yutong Dai, Yuhao Yang, Ziyang Luo, Zirui Zhao, Zhiyuan Hu, Junzhe Huang, Amrita Saha, Zeyuan Chen, et al. Gta1: Gui test-time scaling agent. *arXiv preprint arXiv:2507.05791*, 2025. 3
- [53] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, et al. Mobile-agent-v3: Fundamental agents for gui automation. *arXiv preprint arXiv:2508.15144*, 2025. 1, 3
- [54] Wenwen Yu, Zhibo Yang, Jianqiang Wan, Sibao Song, Jun Tang, Wenqing Cheng, Yuliang Liu, and Xiang Bai. Omniparser v2: Structured-points-of-thought for unified visual text parsing and its generality to multimodal large language models. *arXiv preprint arXiv:2502.16161*, 2025. 5
- [55] Junlei Zhang, Zichen Ding, Chang Ma, Zijie Chen, Qiushi Sun, Zhenzhong Lan, and Junxian He. Breaking the data barrier—building gui agents through task generalization. *arXiv preprint arXiv:2504.10127*, 2025. 3
- [56] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023. 3